

Team Jailhouse Lock

Josue Peraza Navarrete, Yash Patel, Derrick Casseus, Emmanuel Martinez, Joseph Falco, Leo Oliveira

PASSWORD MANAGER APP



Software Requirements and Specifications

Table of Contents

1. Introduction.....	1
1.1 Purpose	
1.2 Document Conventions	
1.3 Scope	
1.4 Intended Audience	
1.5 Definitions, Acronyms, Abbreviations	
1.6 Software Licensing	
1.7 References	
2. System Overview	7
2.1 System Perspective	
2.2 System Functionality	
2.3 System Constraints	
2.3.1 Offline Functionality & Local Storage	
2.3.2 Secure Access	
2.3.3 Web-Based Synchronization	
2.3.4 Database Security, Scalability and Maintenance	
3. Functional Requirements.....	9
3.1 User Authentication	
3.2 Password Storage & Management	
3.3 Autofill & Integration	
3.4 Password Generator	

3.5 Password Checker	
3.6 Data Backup & Import/Export	
3.7 Security & Encryption	
4. System Features.....	11
4.1 Secure Usage	
4.1.1 Multi-factor Authentication	
4.1.2 Security Alerts	
4.2 Data Storage	
4.2.1 Credentials Storage	
4.2.2 Device ID Storage	
4.3 Threat Scanning	
4.3.1 Monitoring for Threat Actors	
4.3.2 Internal Testing	
5. User Interface And Conceptual Model.....	15
5.1 Mockup UI	
5.2 Password View	
5.3 Security View	
5.4 Settings	
5.5 Profile View	
6. Other Nonfunctional Requirements.....	16
6.1 Security Requirements	
6.2 Performance Requirements	
6.3 Usability and Availability	

1. Introduction

1.1 Purpose

The purpose of this document is to describe in detail the technical and functional requirements necessary that GateKeep has and essentially what it does and what it's for. GateKeep is a cross-platform password manager desktop app designed to securely store, manage, generate, and autofill user credentials across different applications and browsers, this application aims to enhance security and convenience for users by providing a **centralized password vault** protected by a **master password** for handled security and preservation.

1.2 Document Conventions

For any future updates or adjustments to this document, modifications **MUST** stick to the following standards:

- Times New Roman Font
- 14-point bolded font for headings
- 13-point bolded font for subheadings
- 12-point not bolded font for any of the other text
- Double-spaced font UNLESS it's a list (like in this case) then it must be 1.15 spaced out

1.3 Scope

GateKeep will provide users with a secure keychain accessible via a master password which allows them to store, retrieve, and manage their credentials such as username and password.

Furthermore, the system will support Windows 11 and macOS leveraging industry-standard

encryption and advanced security practices. In the future of the application, GateKeep will integrate with web browsers and applications for seamless login experiences and includes cloud synchronization capabilities for cross-device accessibility allowing for more ease of access for a user's passwords.

1.4 Intended Audience

- Users who desire a secure platform to store their passwords in a location that can be accessed by multiple different browsers and applications across their device(s) by just entering a master password.
- Users who tend to create weak passwords and bad password-making tendencies and wish to create safe and secure passwords using GateKeep's password checker.
- Users who worry about their security and want to generate secure passwords with the advantage of having access to all of them by just memorizing one single password (the master key).

1.5 Definitions, Acronyms, and Abbreviations

- **Git** - Git is a distributed version control system for tracking changes in source code during the software development process.
- **Github** - A global company that provides hosting for software development version control using Git.
- **GUI** - Graphical User Interface
- **Open Source Software** - Refers to a piece of software that people can modify and share because it's designed to be of public access.
- **AES-256** - Advanced Encryption Standard (256-bit) for data encryption.
- **Argon2** - Key derivation function used for hashing the master password.
- **PostgreSQL** - Cloud-based database used for storing encrypted credentials.
- **Credential Manager API** - Windows API for storing credentials securely.
- **Keychain API** - macOS secure storage mechanism.
- **Secret Service API** - Linux secure credential storage.

- **Passlib.py** - Library for password hashing and security functions.
- **MSIX, AppImage, and DMG** - These are formats for distributing the application on Windows, Linux, and macOS, respectively.

1.6 Software Licensing

GateKeep will be released under the GNU General Public License (GPL) v3.0 which ensures that the software remains free and open-source while granting users the rights of:

- Freedom to use it, run it, study it, and modify the software for any purpose under ethical standards.
- Freedom to distribute the software, including modified versions under the same GPL license.
- Improvements remain open to the community due to copyleft protection.

1.7 References

- [https://www.geeksforgeeks.org/how-to-build-a-password-manager-in-python/lesspass/lesspass: :key: stateless open source password manager](https://www.geeksforgeeks.org/how-to-build-a-password-manager-in-python/lesspass/lesspass%3A%3Akey%3Astateless+open+source+password+manager)
- [Build cross-platform desktop apps with JavaScript, HTML, and CSS | Electron](#)
- [Node.js — Run JavaScript Everywhere](#)
- [Azure Database for PostgreSQL – Database Cloud | Microsoft Azure](#)
- [PostgreSQL: Documentation: 16: 18.1. Building with Visual C++ or the Microsoft Windows SDK](#)
- [Advanced Encryption Standard \(AES\) - GeeksforGeeks](#)
- [Argon2 - Wikipedia](#)
- [Credentials Management - Win32 apps | Microsoft Learn](#)
- [Keychain services | Apple Developer Documentation](#)
- [Secret Service API \(Emacs auth-source Library 0.3\)](#)
- [Passlib 1.7.4 documentation — Passlib v1.7.4 Documentation](#)

2. System Overview

2.1 System Perspective

GateKeep can be used to securely store, manage, and retrieve credentials in an organized manner, ensuring easy access to all connected websites, accounts or browsers. It provides a balance of security and convenience, allowing users to quickly autofill login credentials while keeping their information safe. With support for offline local storage and optional web-based synchronization, GateKeep offers flexibility to suit different user needs. Strong encryption, a master password, and potential Two-Factor Authentication (2FA) enhance security, while an intuitive interface makes password management simple for everyone through guiding the user seamlessly through the device and its features. Designed for ease of use, GateKeep ensures a seamless experience while maintaining strong security for the future.

2.2 System Functionality

GateKeep provides a comprehensive password management system for users who need to securely store, manage, and retrieve their credentials. Users will be able to and expand upon these features:

- **Create and manage multiple password entries** for various accounts.
- **Autofill login credentials** when accessing websites and applications.
- **Generate strong passwords** and replace weak/reused ones.
- **Organize passwords into categories** for easier management.
- **Password recovery** after a password has already been deleted.
- **Monitor password health** with built-in security analysis tools.

- **Store and retrieve secure notes and personal information** beyond passwords.
- **Receive real-time security alerts** for potential password breaches.

2.3 System Constraints

GateKeep is designed to have multiple uses for its user interface as well as different constrictions to have a more secure experience for the user and to ensure top support with future updates and processes that will increase the use of the software for all users collectively.

2.3.1 Offline Functionality & Local Storage

GateKeep works offline, storing passwords securely on the user's device. This ensures access without needing an internet connection and reduces exposure to online threats.

2.3.2 Secure Access

Users log in with a master password, with the future option to enable Two-Factor Authentication (2FA) for added security. This prevents unauthorized access even if the master password is compromised.

2.3.3 Database Security, Scalability and Maintenance

Ensures strong database security by encrypting stored passwords, populating the stack, and preventing unauthorized access or data exposure. The system is also designed with scalability and maintainability in mind, allowing for updates, feature additions, and security improvements without affecting the overall functionality.

3. Functional Requirements

3.1 User Authentication

Users must enter a **master password** to unlock their credentials. The master password is hashed using **Argon2** before storage. GateKeep will commence **auto-logout** after a period of inactivity.

3.2 Password Storage & Management

Passwords are encrypted using **AES-256** before being stored in **PostgreSQL**. Users can **add, edit, delete, and organize** stored credentials in the application. The passwords will be displayed along with the respective website on the main page of the application. The application checks the input passwords for qualities indicating weak and possibly unsafe passwords. **Cloud Sync** will allow for seamless access across multiple devices so that the application can connect to a specific account and the passwords stored on it using the **master password**.

3.3 Autofill & Integration

GateKeep will integrate with the **Credential Manager API (Windows)** and **Keychain API (macOS)**. This allows GateKeep to utilize its stored passwords across multiple different applications. GateKeep will allow the user to copy credentials from the application into the used browser.

3.4 Password Generator

The password generator gives users the ability to create passwords when users can't create a strong password on their own. Strong, unique passwords are generated using **cryptographic**

randomization. The password generator offers suggestions to users based on parameters given by the user. Parameters of the generator include password length, symbols, and complexity. All given are generated based on the password checker requirements for a strong password.

3.5 Password Checker

Checks current passwords based on the following parameters:

Strong	Weak
• 8 - 16 characters • Good mix of uppercase and lowercase letters • Include numbers and other special characters (!,@, #, etc.)	• Repeated Passwords • Personal information (username, date of birth) • Simple and/or common phrases (password, 1234, etc.)

When weak passwords are detected:

- On the password tiles, a red error symbol indicating a weak password detected will be displayed.
- When the tile is clicked a pop-up gives users information indicating how the password was seen as weak and what it's missing to become a strong password.
- If the application is not open while the password is detected as weak, a notification will pop up on the desktop through a Google extension with bars indicating how weak/strong the password is.

3.6 Data Backup & Import/Export

GateKeep will provide **encrypted backup options** for password storage. Users will be allowed to **import/export credentials** in a secure format that is encrypted and password-enforced. **Cloud recovery options** will be available for lost local files. This includes backups in the cloud and through user authentication.

3.7 Security & Encryption

GateKeep uses **AES-256** encryption for storing passwords. The master password will be secured using **Argon2 hashing**. Encrypted credentials will be in **PostgreSQL**. GateKeep will have real-time alerts in cases of potential **security breaches**. When an alert is raised the application will allow users to swiftly change the compromised password and any other passwords that may also raise concerns.

4. System Features

The following is a list of features incorporated into the GateKeep system. These features are arbitrarily listed and not in any order of ranked importance. This list will be updated to include any additional features given that more features are included with GateKeep in the future.

4.1 Secure Usage

4.1.1 Multi-factor Authentication

This application requires that users provide two means of authenticating themselves before allowing access to their credentials stored in GateKeep. The first method of authentication involves the use of a username and password specifically created and used by a given user. Once

an account is created, the user must also create an additional master password to be used to complete the multifactor authentication process.

- With this master password, the user has to use it each time they log in to GateKeep on an unregistered device. After this secondary password is used, the user will be prompted to register this device as one of their own. If the user does register the currently used device, the ID associated with this device will be moved into a separate database inside of GateKeep to be recognized later on. In cases where the device isn't registered, the user will have to go through the process of 2FA again.

4.1.2 Security Alerts

In the case where there have been multiple failed attempts to log into a user's account consecutively, the device that's logging into GateKeep will be temporarily locked out. At the same time, GateKeep will create an alert for the intended user to see when they next login that someone attempted to break into their account. Afterward, the user is suggested to update their current credentials to prevent any future unauthorized logins.

4.2 Data Storage

4.2.1 Credentials Storage

GateKeep contains an entire database dedicated to the storage of all passwords. The database is inaccessible to all viewers through utilizing PostgreSQL's option to remove the key needed to access the database. The passwords are matched with their respective credentials only within the

database and not outside, preventing hackers from easily obtaining credentials just by finding the location of stored usernames.

4.2.2 Device ID Storage

GateKeep also utilizes a separate database meant to host the IDs of all devices registered to a specific GateKeep account. The IP address of the device is not kept in storage. Instead, a specific ID created upon the registration of a device is kept stored to prevent privacy threats in the case where the database is broken into. The IDs are encrypted using AEGIS-256 with their own set of keys.

4.3 Threat Scanning

4.3.1 Monitoring For Threat Actors

GateKeep performs a daily internal scan to look for any signs of threats or interference from outside forces. In the case that a threat has been identified, the user whose copy of GateKeep is being threatened will be alerted about the threat their copy is facing. The alert will suggest that the user change their password alongside contacting JailHouse Locke with the provided contact information.

- GateKeep alerts the user also when there is a failure in one of its key functions. The alert will inform the user of what exactly went wrong without forcing the user to quit their current process.

4.3.2 Internal Testing

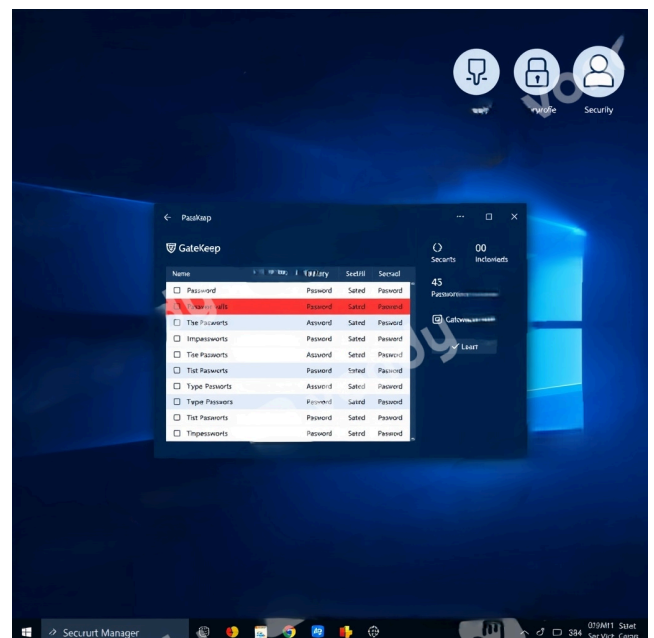
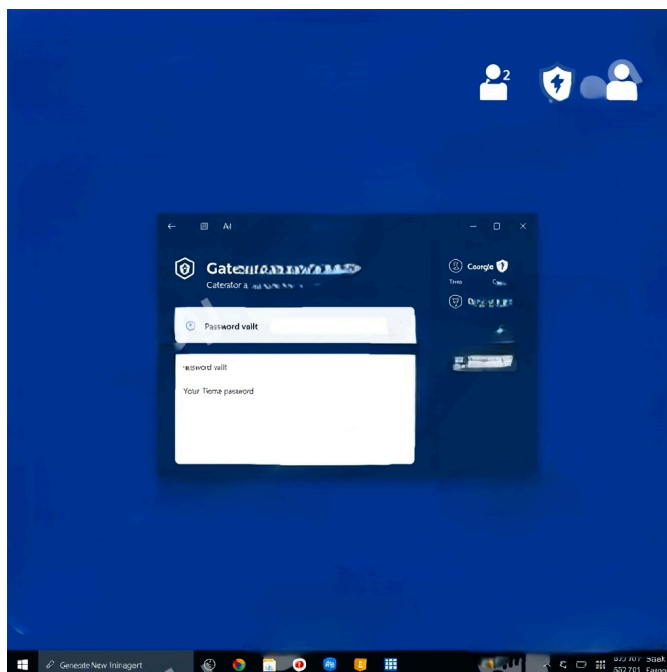
- JailHouse Locke performs tests on GateKeep to ensure GateKeep has not been tampered with or infected with a virus. GateKeep performs penetration tests to attempt to break

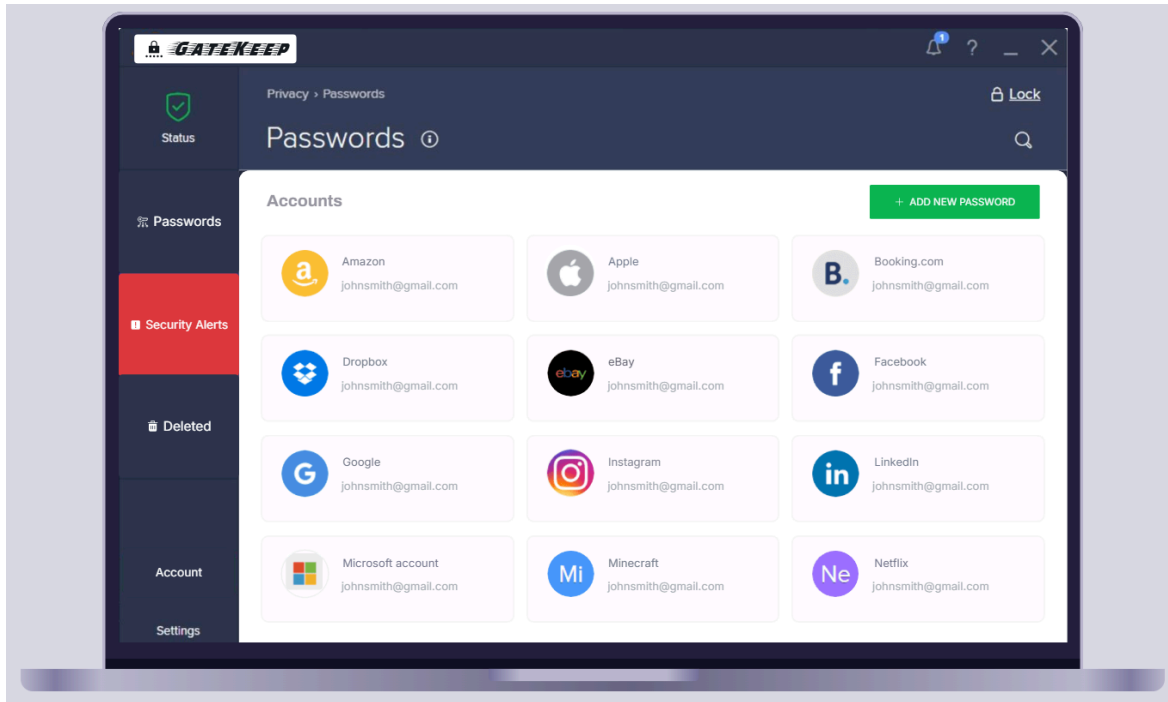
through GateKeep's defenses. If any penetration tests succeed, GateKeep will record that a test failed alongside the method used to penetrate GateKeep's defenses. From there, JailHouse Locke will allocate resources to promptly remove the flaw in GateKeep's programming.

5. Graphical User Interface And Conceptual Model

5.1 Mockup UI

- Below are some potential models of the application which may be subject to change.





On the left-hand side is a vertical menu with options such as passwords, security checks, and settings. The menu also contains a circular profile image which is also a button to allow users to modify their personal information. The right side which is the majority of the window, will display the information of whatever menu section the user is currently on.

5.2 Password View

Passwords are displayed as tiles and can be sorted in multiple ways such as newest first, oldest first, or alphabetically. In the top right corner, there is a search button that the user uses to search for a specific saved password. Underneath the search button, there is also a button for users to manually add passwords and save them to the password manager. Each tile will display the image and name of the service as the heading and the username underneath. When the user interacts with the tile, it will become a small pop-up within the application. This pop-up contains the image and name of the service on top left and below it is the username and the password. The

username is in plain text and the password is masked with the ability to turn to plain text when the user hovers over it. Both fields contain an option to copy the text.

5.3 Security View

Each password saved is checked for reuse and displayed if there is a match for multiple occurrences. A warning message alerting the user is displayed on top of the list of passwords reused stating how many times a particular password has been reused. In the list of passwords, each has an option to change the password for more security.

5.4 Settings

There are settings in the form of a list. The first toggle allows the user to switch between dark and light modes. The second toggle asks the user if the application can save passwords automatically. Next, there is an option for the user to export the saved passwords as a CSV.

6. Other Nonfunctional Requirements

6.1 Security Requirements

Users will automatically be logged out after being inactive for a minute. This protects sensitive information in case the user leaves their device unattended for better security purposes and preservation of private data. Security is our primary concern with developing GateKeep. A password manager without security is not a quality product. Users do not need to run GateKeep

in Administrator mode to utilize it. GateKeep encrypts passwords using AEGIS-256 and securely stores those encrypted passwords in our database. GateKeep will never store or access a user's master password. GateKeep will provide error messages and logs for debugging.

6.2 Performance Requirements

- GateKeep needs to meet the basic requirements of the user's operating system to run.
- GateKeep performs with **little to no latency** when interacting with the software.
- GateKeep can efficiently handle many users without loss of performance.

6.3 Usability and Availability Requirements

- GateKeep has a **simple and intuitive** user interface for any user.
- GateKeep's cloud aims to achieve **99.9% uptime**.
- Users can access their stored passwords **offline**.
- GateKeep currently supports Windows and macOS and will consider adapting to mobile devices in the future